# Industrial Internship Report on

# "FreshCart — Full Stack Grocery Delivery Web Application"

## Prepared by

## DIKSHA

| *Executive Summary* |
| --- |
| This report provides details of the Industrial Internship offered by Upskill Campus and The IoT Academy, in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT). This internship was structured around a project/problem statement I selected within the Full Stack Web Development track.<br><br>The complete project, including this report, was carried out over one month. My project is FreshCart, a full stack grocery delivery web application built with a genuine three-tier architecture: an HTML/CSS/JavaScript frontend, a Java Servlet + JDBC backend, and a MySQL database. The application supports user signup and login, product browsing with search and category filters, cart management, checkout with delivery scheduling, and order history — all backed by persistent, relational storage rather than simulated client-side data.<br><br>This internship gave me my first real, hands-on exposure to industry-style Java web development — including setting up and debugging Eclipse, Apache Tomcat, and MySQL from scratch. It was a genuinely challenging but rewarding experience that significantly strengthened my confidence in learning and deploying an unfamiliar technology stack independently. |

**TABLE OF CONTENTS**

# 1 Preface

This report summarizes the work completed during a one-month “Full Stack Web Development” internship offered through Upskill Campus (USC) in association with UniConverge Technologies (UCT). As a first-year BCA student at World College of Technology & Management, this internship was my first structured, hands-on exposure to industry-style software development, giving me a chance to move beyond classroom theory into building something functional from start to finish.

Relevant internships like this matter enormously for career development because they close the gap between what is taught in a classroom and what is actually expected in a real development environment — configuring tools, debugging real errors, and making independent decisions under a deadline.

Coming in with only prior experience in HTML, CSS, and vanilla JavaScript, the internship's weekly structure — progressing from Java fundamentals (OOP, classes, inheritance, polymorphism) through JDBC and finally into a live web project — pushed me to learn an entire new stack (Java Servlets, JDBC, MySQL, Apache Tomcat, Eclipse IDE) within a short timeframe.

The project I chose was FreshCart, a grocery delivery web application, built with a genuine three-layer architecture: an HTML/CSS/JavaScript frontend, a Java Servlet + JDBC backend, and a MySQL database — the same pattern used in real enterprise Java web applications, rather than a simplified, simulated version.

The program was planned across four weeks: Week 1 covered HTML/CSS/JS fundamentals and project planning, Week 2 covered Java and Object-Oriented Programming, Week 3 introduced JDBC and database connectivity, and Week 4 was dedicated to building, deploying, and testing the complete full-stack application.

Over these weeks, my biggest learning was not any single line of code, but the discipline of debugging methodically — reading a Java stack trace properly, isolating one variable at a time, and not giving up when the first fix does not work.

I would like to thank Upskill Campus and UniConverge Technologies for structuring this internship in a way that balanced guided learning with independent problem-solving. Setting up and debugging a full Java web environment for the first time was genuinely challenging, but that struggle is where most of the learning happened.

To juniors and peers who will take on a similar internship: don't be discouraged by setup errors and configuration issues — a blank Tomcat 404 page or a MySQL access-denied error is a completely normal part of learning a new backend stack, and debugging them teaches you more than a smooth run ever would.

# 2 Introduction

## 2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.

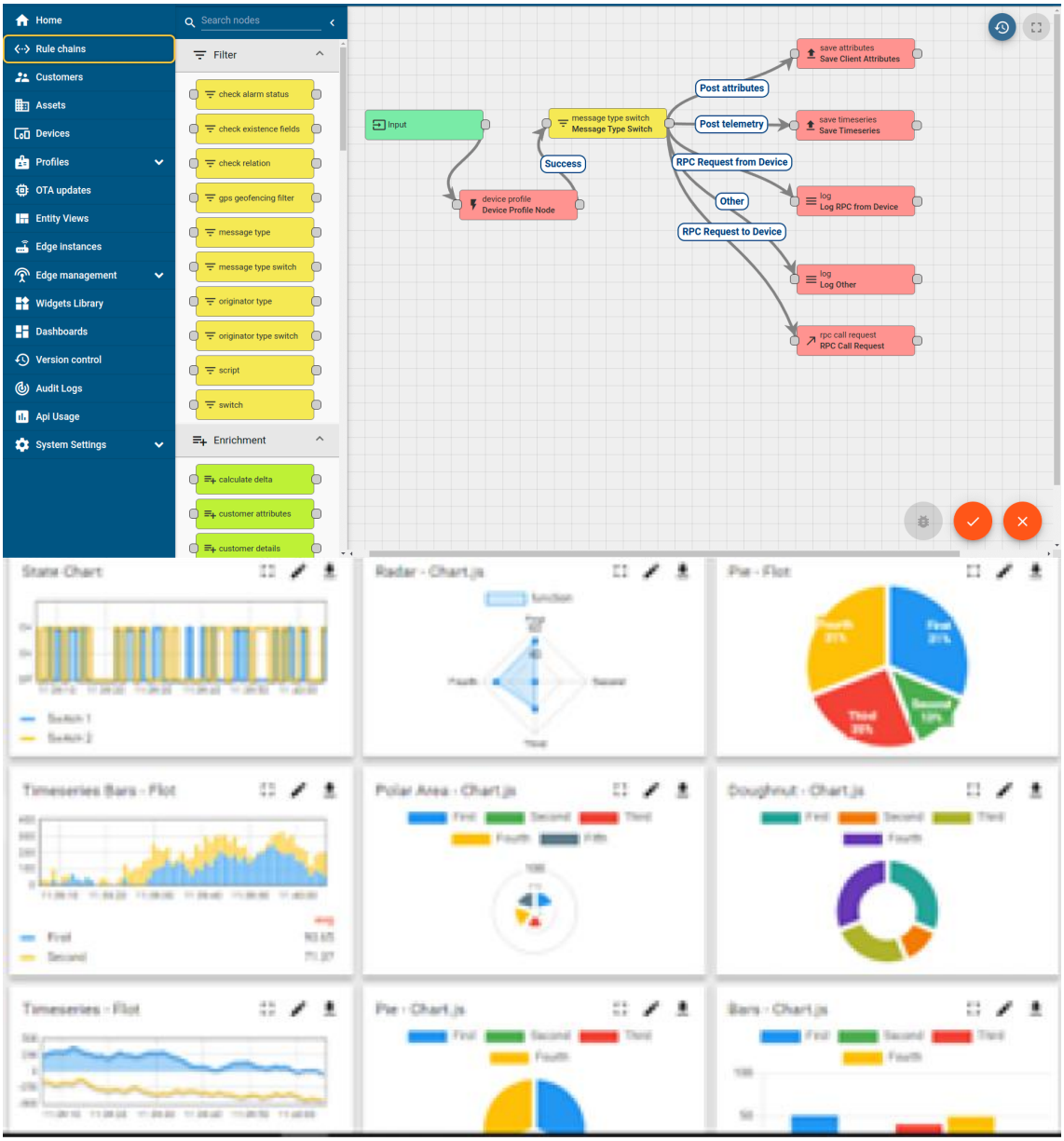

## i. UCT IoT Platform (UCT Insight)

**UCT Insight** is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable "insight" for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine



## ii. Smart Factory Platform ( FACTORY WATCH )

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleased the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.

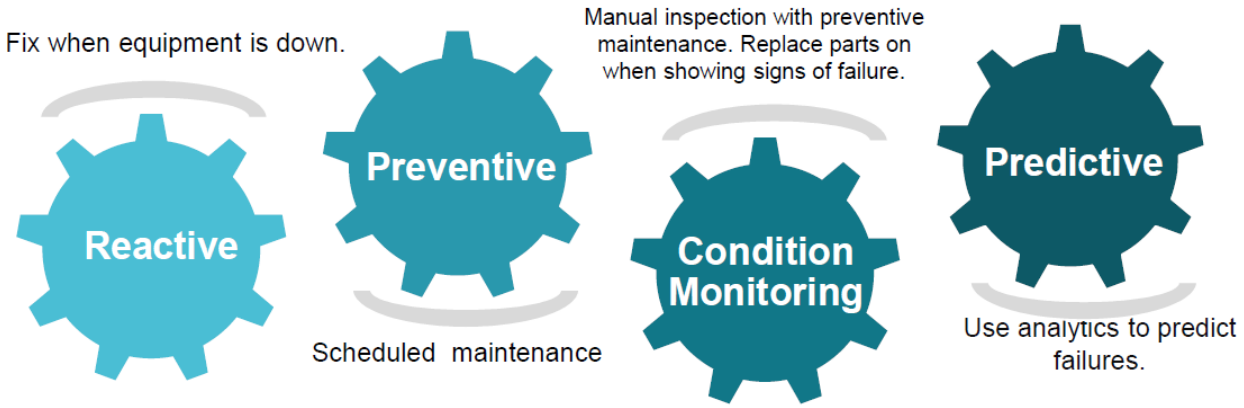

| Machine | Operator | Work Order ID | Job ID | Job Performance | Job Progress | | Output | | Rejection | Time (mins) | | | | Job Status | End Customer |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | Start Time | End Time | Planned | Actual | | Setup | Pred | Downtime | Idle | | |
| CNC_S7_81 | Operator 1 | WO0405200001 | 4168 | 58% | 10:30 AM | | 55 | 41 | 0 | 80 | 215 | 0 | 45 | In Progress | i |
| CNC_S7_81 | Operator 1 | WO0405200001 | 4168 | 58% | 10:30 AM | | 55 | 41 | 0 | 80 | 215 | 0 | 45 | In Progress | i |

## iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

## iv. Predictive Maintenance

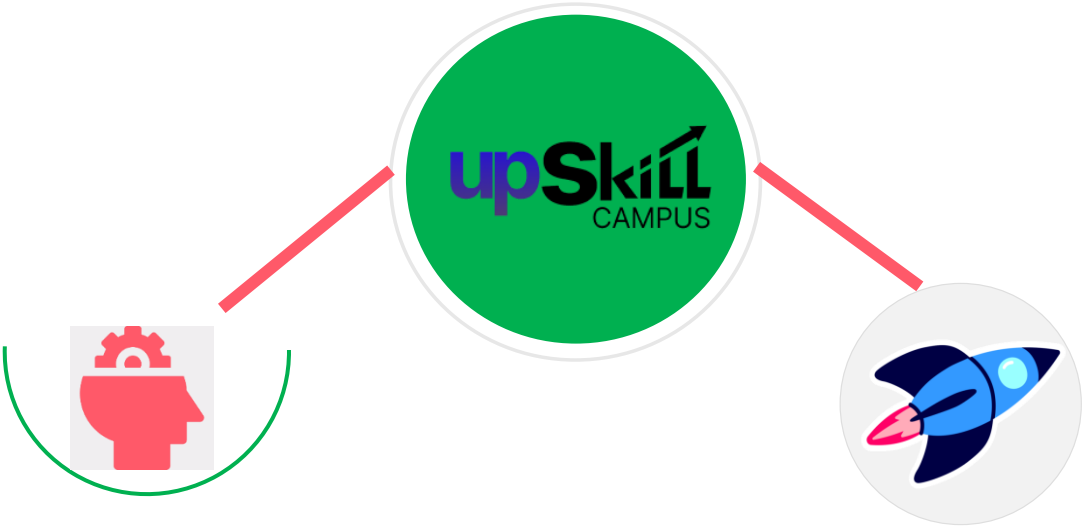
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.




### 2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

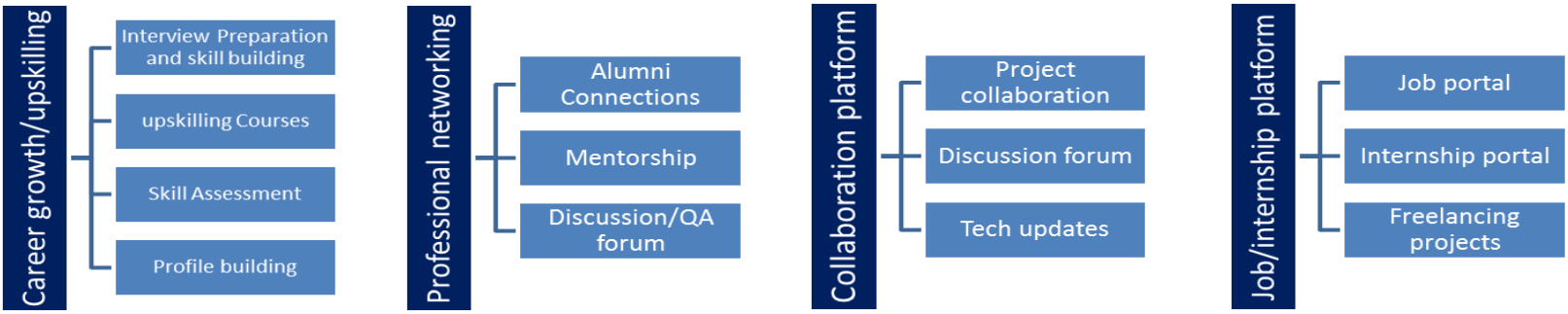
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.




Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

https://www.upskillcampus.com/



## 2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

## 2.4 Objectives of this Internship program

The objective for this internship program was to

☛ Get practical, hands-on experience of working with a real-world Java full-stack development environment.

☛ Learn to solve real problems by designing and building a complete application end-to-end, not just isolated coding exercises.

☛ Build a foundation in Java web development (Servlets, JDBC) to strengthen future job prospects.

☛ Gain an improved understanding of how frontend, backend, and database layers connect and communicate in a real application.

☛ Develop personal growth in communication, patience, and systematic problem-solving under real deployment constraints.

## 2.5 Reference

[1] Upskill Campus — Full Stack Web Development Internship course material (Weeks 1–4).

[2] Oracle Java Servlet and JDBC official documentation.

[3] MySQL 8.0 Reference Manual and Apache Tomcat 9 official documentation

## 2.6 Glossary

| Terms | Meaning |
|---|---|
| Servlet | A Java class that handles HTTP requests and responses on a web server. |
| JDBC | Java Database Connectivity — the standard Java API for connecting to and querying relational databases. |
| DAO | Data Access Object — a design pattern isolating all database code into dedicated classes. |
| CORS | Cross-Origin Resource Sharing — a browser rule this project's CorsFilter explicitly permits so the frontend can call the backend. |
| Tomcat | The Apache servlet container used to run and deploy the Java backend. |

# 3 Problem Statement

Grocery shopping through a physical store, or through a poorly designed website, is often inconvenient. Customers expect to browse a catalog, search and filter products by category, manage a cart, and check out with a chosen delivery slot — all without friction, and with confidence that their order is actually saved.

The assigned problem was to design and build a functioning grocery e-commerce web application supporting account creation, product browsing with search/filtering, cart management, checkout with delivery scheduling, and order history — backed by a real, persistent database rather than temporary or simulated (client-only) storage. This is a deliberately realistic problem: it mirrors the structure of any small e-commerce backend a junior Java developer might be asked to build or maintain.

# 4 Existing and Proposed solution

Existing commercial grocery delivery platforms solve this problem at scale, but their codebases are closed source, architecturally complex, and inaccessible for learning purposes. At the other end, most introductory student projects simulate a backend using only browser localStorage, which does not teach real server-side development, relational database design, authentication, or client-server communication — the exact skills this internship's Java + JDBC curriculum was designed to build.

My proposed solution, FreshCart, deliberately avoids both extremes. It is a genuine three-tier application:

● Frontend: Seven HTML/CSS/JavaScript pages (Home, Cart, Checkout, Orders, Login, Signup) that communicate with the backend exclusively through fetch() and standard HTTP requests — never touching the database directly.

● Backend: A Java Servlet-based API (SignupServlet, LoginServlet, ProductServlet, OrderServlet), using JDBC and a dedicated DAO layer to talk to MySQL.

● Database: A MySQL schema with four related tables — users, products, orders, and order_items — connected via foreign keys, enforcing referential integrity at the database level.

The value addition of this solution, relative to a purely client-side project, is that it demonstrates genuine understanding of server-side architecture: authentication, persistent storage, transactional order placement (so an order is never left half-saved), and separation of concerns via the DAO pattern.

## 4.1 Code submission (Github link)

https://github.com/dikshatehdpuriya/upskillcampus/tree/main/com/freshcart

## 4.2 Report submission (Github link)

https://github.com/dikshatehdpuriya/upskillcampus/blob/main/FreshCart_Diksha_USC_UCT.pdf

# 5 Proposed Design/ Model

The application follows a standard three-tier web architecture. The browser (frontend) never talks to the database directly — it always goes through the Java backend, which is the only layer holding database credentials and query logic. This separation is a core principle of secure, maintainable web application design, and is reflected directly in the package structure of the codebase.

## 5.1 High Level Diagram

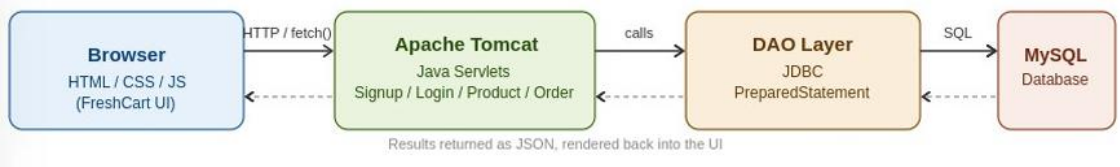



**Figure 1: HIGH LEVEL DIAGRAM OF FRESHCART SYSTEM**

At the highest level, a request starts in the browser, travels over HTTP to Apache Tomcat, is routed to the matching Servlet, which delegates all database work to the DAO layer, which in turn executes SQL against MySQL. The response follows the same path in reverse, arriving back at the browser as JSON that the frontend renders into the page.

## 5.2 Low Level Diagram

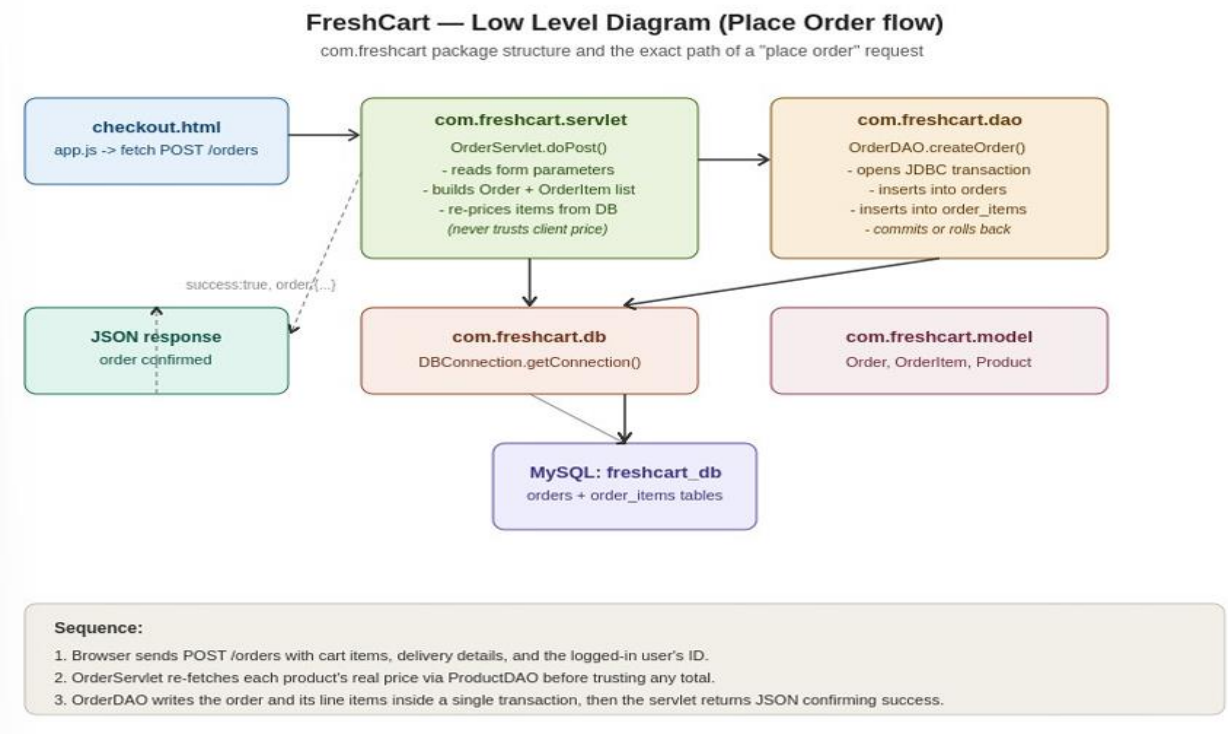



**Figure 2: Detailed "place order" data flow through the com.freshcart package structure**

The backend is organized into six packages under com.freshcart, each with a single responsibility:

● model — User, Product, Order, OrderItem: plain Java objects representing each database table's rows, each with a toJson() method for building API responses.

● dao — UserDAO, ProductDAO, OrderDAO: all JDBC/SQL logic (PreparedStatement, ResultSet, transactions) lives here, isolated from the web layer.

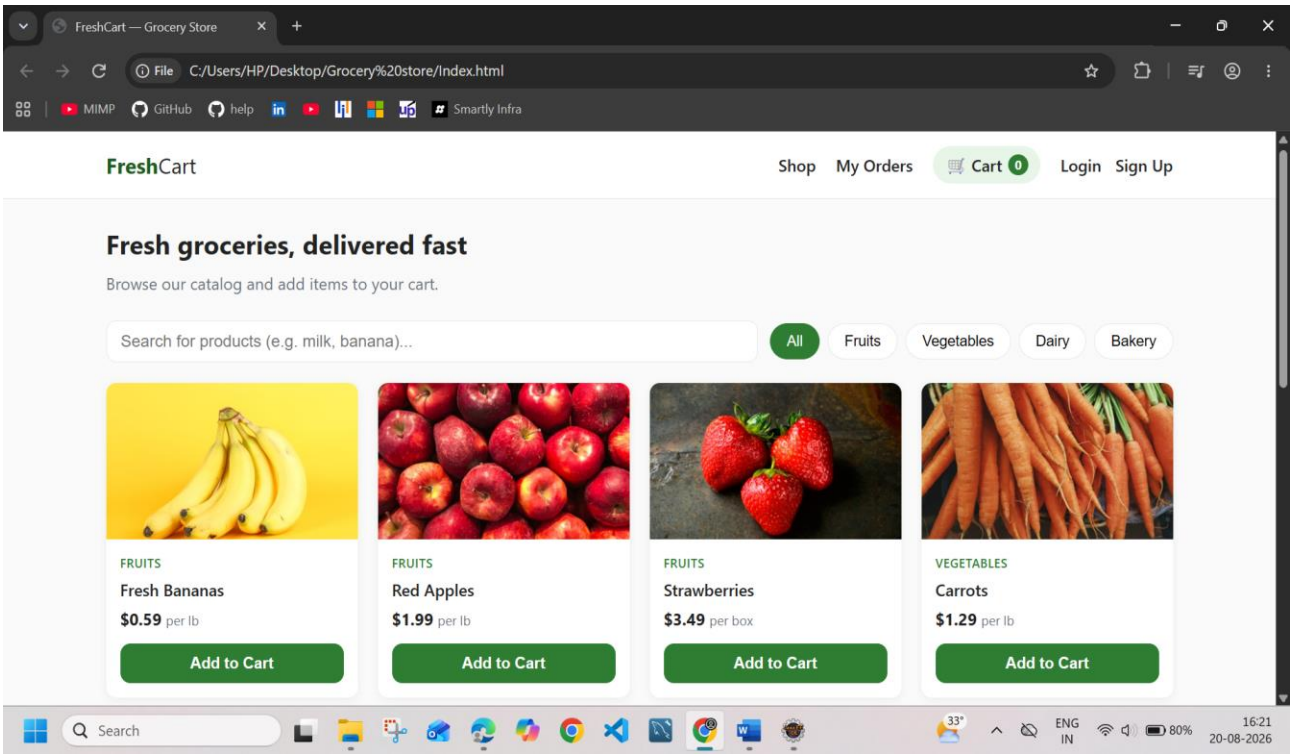
● db — DBConnection: a single shared method that opens a JDBC connection to MySQL, reading credentials from an external db.properties file rather than hardcoding them in source code.

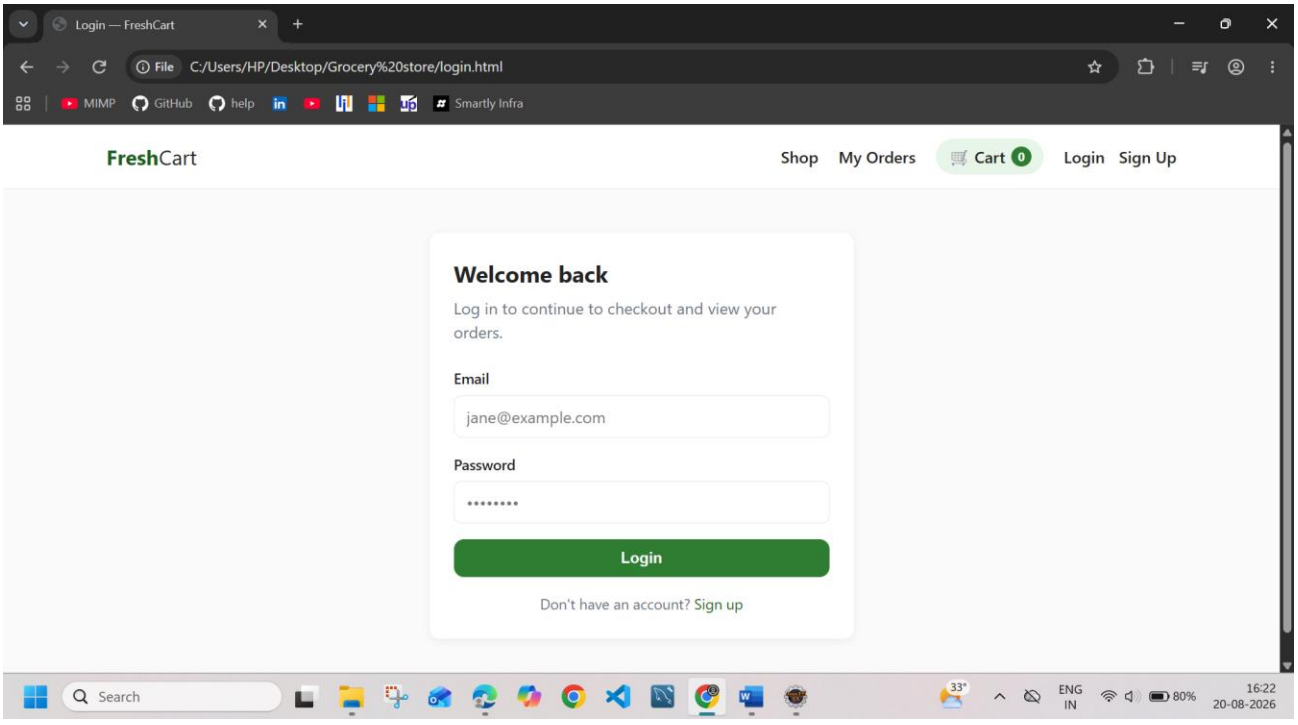
● servlet — SignupServlet, LoginServlet, ProductServlet, OrderServlet: the HTTP-facing layer, each mapped to a URL (e.g. /products, /orders) via the @WebServlet annotation.

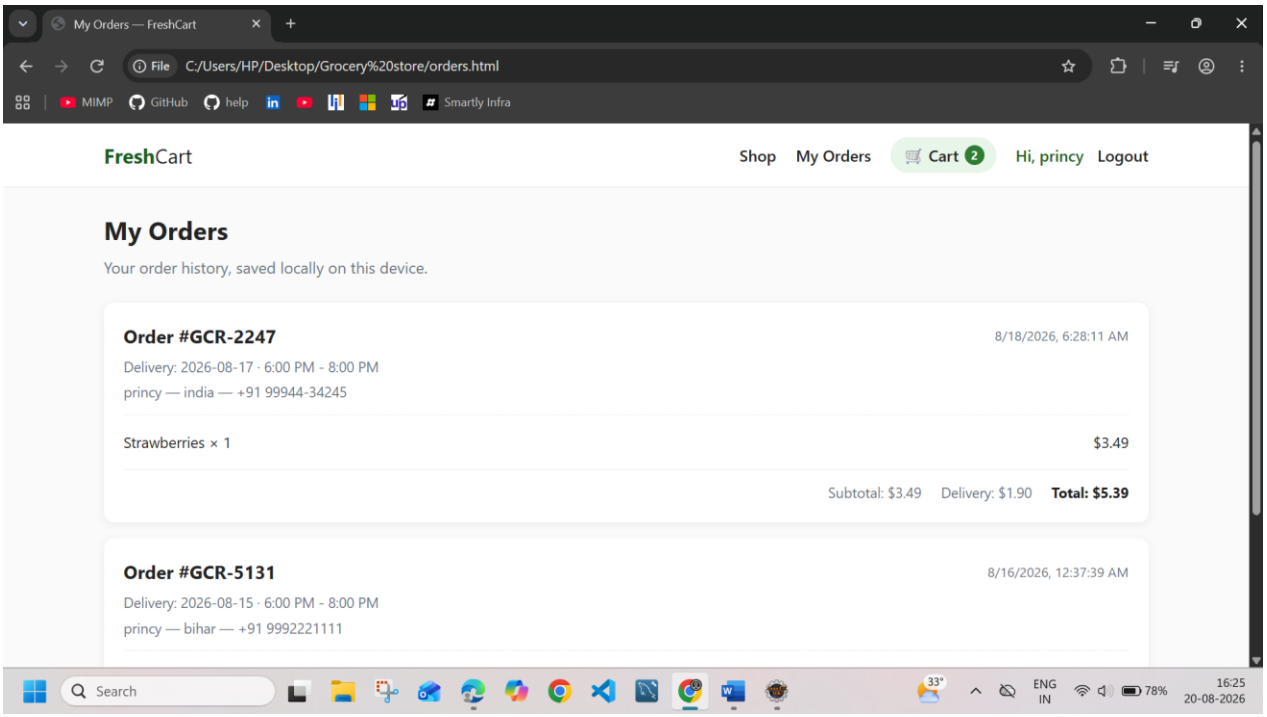
● filter — CorsFilter: adds CORS headers so the frontend, opened as a separate origin during development, is permitted to call the backend.

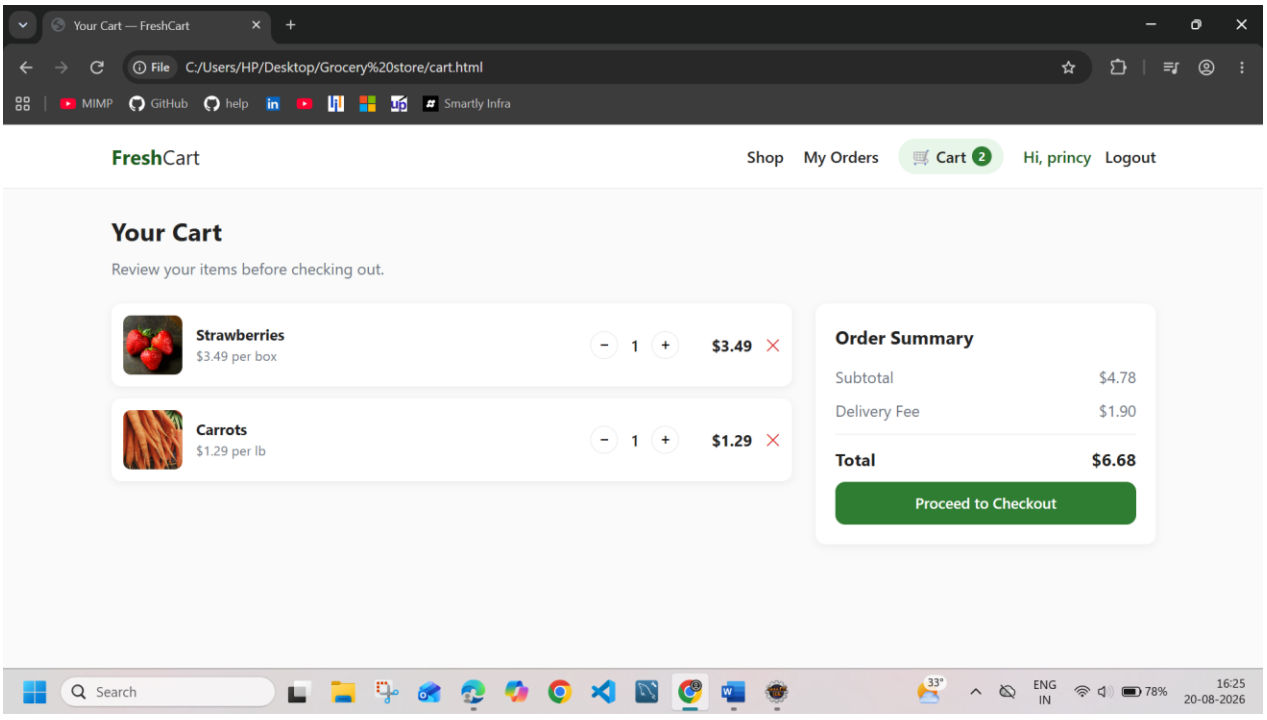
● util — JsonUtil: a small hand-written helper for safely building and escaping JSON strings, avoiding an external dependency.

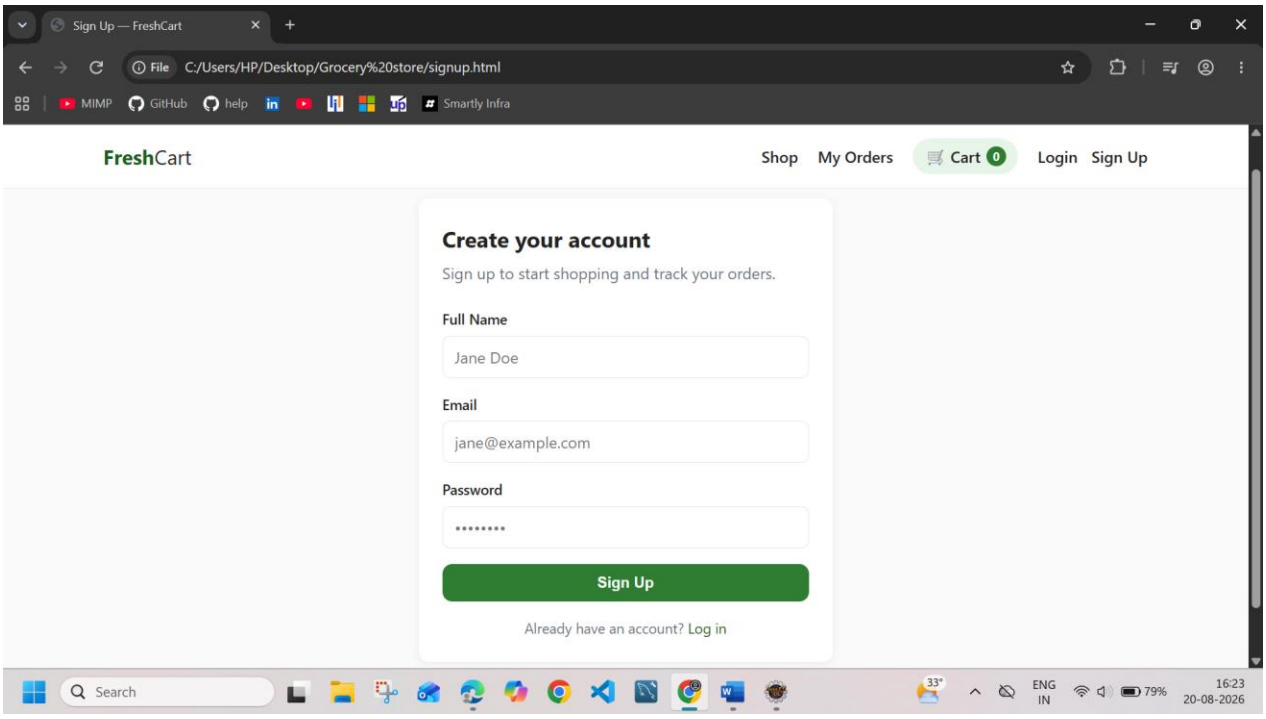
## 5.3 Screenshots of the Working Application

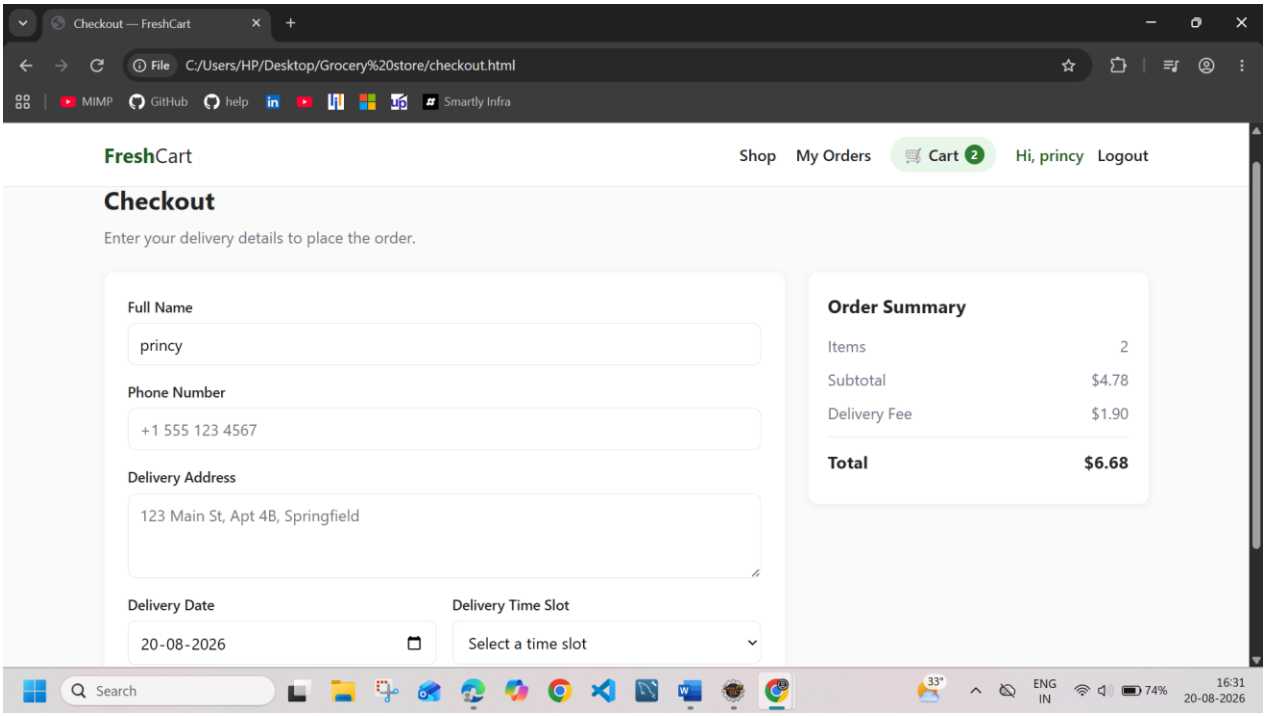



My Orders — FreshCart
File C:/Users/HP/Desktop/Grocery%20store/orders.html
MIMP GitHub help Smartly Infra
FreshCart
Shop My Orders Cart 2 Hi, princy Logout
My Orders
Your order history, saved locally on this device.
Order #GCR-2247
8/18/2026, 6:28:11 AM
Delivery: 2026-08-17 · 6:00 PM - 8:00 PM
princy — india — +91 99944-34245
Strawberries × 1
$3.49
Subtotal: $3.49 Delivery: $1.90 Total: $5.39
Order #GCR-5131
8/16/2026, 12:37:39 AM
Delivery: 2026-08-15 · 6:00 PM - 8:00 PM
princy — bihar — +91 9992221111
Search
ENG IN
78%
16:25
20-08-2026


Your Cart — FreshCart
File C:/Users/HP/Desktop/Grocery%20store/cart.html
MIMP GitHub help Smartly Infra
FreshCart
Shop My Orders Cart 2 Hi, princy Logout
Your Cart
Review your items before checking out.
Strawberries
$3.49 per box
1
$3.49
Carrots
$1.29 per lb
1
$1.29
Order Summary
Subtotal $4.78
Delivery Fee $1.90
Total $6.68
Proceed to Checkout
Search
ENG IN
78%
16:25
20-08-2026


Sign Up — FreshCart
File C:/Users/HP/Desktop/Grocery%20store/signup.html
MIMP GitHub help Smartly Infra
FreshCart
Shop My Orders Cart 0 Login Sign Up
Create your account
Sign up to start shopping and track your orders.
Full Name
Jane Doe
Email
jane@example.com
Password
Sign Up
Already have an account? Log in
Search
ENG IN
79%
16:23
20-08-2026


Checkout — FreshCart
File C:/Users/HP/Desktop/Grocery%20store/checkout.html
MIMP GitHub help Smartly Infra
FreshCart
Shop My Orders Cart 2 Hi, princy Logout
Checkout
Enter your delivery details to place the order.
Full Name
princy
Phone Number
+1 555 123 4567
Delivery Address
123 Main St, Apt 4B, Springfield
Delivery Date
20-08-2026
Delivery Time Slot
Select a time slot
Order Summary
Items 2
Subtotal $4.78
Delivery Fee $1.90
Total $6.68
Search
ENG IN
74%
16:31
20-08-2026

# 6 Performance Test

As a learning-focused internship project running on a local development machine, formal load or performance testing (concurrent users, high request volume) was outside the project's scope. Instead, testing focused on functional correctness and on identifying real deployment constraints — which, in practice, is where most of the debugging effort in this project actually went. Constraints considered: correctness of the JDBC connection configuration, correct servlet-to-URL mapping under Tomcat, and correct transactional behaviour when saving an order across two related tables. Each of these was tested directly rather than assumed.

## 6.1 Test Plan/ Test Cases

| # | Test Case | Expected Result | Status |
|---|---|---|---|
| 1 | Signup with a new email address | Account created in users table; session established | Pass |
| 2 | Signup with an already-registered email | Rejected with a clear duplicate-account error | Pass |
| 3 | Login with correct credentials | Redirects to home page; nav shows the user's name | Pass |
| 4 | Login with incorrect password | Rejected with an "incorrect email or password" message | Pass |
| 5 | Product search and category filter | Correct subset of products returned by a live SQL query | Pass |
| 6 | Add / update / remove cart items | Subtotal and delivery fee recalculate correctly | Pass |
| 7 | Checkout while not logged in | Redirected to login, then back to checkout after login | Pass |
| 8 | Place an order | Order + line items saved in one transaction; cart cleared | Pass |
| 9 | View order history | Only the logged-in user's own past orders are shown | Pass |

## 6.2 Test Procedure

Each feature was tested manually end-to-end: performing the action in the browser, then verifying the result both in the UI (e.g. the Orders page showing a new order with a success banner) and directly in the database via MySQL Workbench (e.g. running SELECT * FROM orders;) to confirm the data was genuinely persisted, not just displayed on screen.

## 6.3 Performance Outcome

A significant real-world constraint encountered was environment configuration rather than application logic. Two issues in particular required deep debugging:

● A recurring Tomcat context-root/deployment mismatch caused repeated HTTP 404 errors on every endpoint, resolved by correcting the project's Web Project Settings context root and forcing a clean redeploy through Eclipse's server configuration.

● A MySQL 8 authentication handshake failure (“Public Key Retrieval is not allowed”) that required adding allowPublicKeyRetrieval=true to the JDBC connection string — a known compatibility requirement between newer MySQL versions and the default JDBC driver settings.

Both issues were diagnosed by reading Tomcat's console stack traces directly rather than guessing, which was itself a genuinely valuable skill built during this project. Once resolved, the application performed correctly and consistently under normal single-user testing conditions, with all nine test cases in Section 6.1 passing.

# 7 My learnings

This project took me from having zero experience with Java web development to independently setting up and debugging a full Eclipse + Tomcat + MySQL environment — tools I had never touched before this internship. Beyond Java syntax itself, I learned:

● How a three-tier web architecture actually works in practice, not just in theory — the exact path a request takes from a browser click to a database row and back.

● Why prepared statements matter for security (preventing SQL injection), and why server-side price validation matters for trustworthiness (never trusting a client-submitted price).

● How to read and interpret a Java stack trace to find the real root cause of an error, rather than guessing or randomly changing settings.

● Practical, professional-grade practices: configuring a servlet container, understanding database transactions and rollback, and separating credentials from source code using an external properties file — a genuinely industry-standard pattern.

● Patience and systematic debugging: several deployment issues (Tomcat 404s, context root resets, MySQL authentication) took multiple attempts to resolve, and I learned to isolate one variable at a time rather than changing many things simultaneously.

This experience directly strengthens my foundation for future coursework and any career path involving backend or full-stack development, and gave me genuine confidence that I can learn and deploy an unfamiliar technology stack independently, without simply following instructions blindly.

# 8 Future work scope

● Deploy the application to a public cloud host (e.g. Render, Railway) with a managed MySQL instance, so it is accessible beyond localhost.

● Add an admin panel for managing the product catalog through the UI, instead of direct SQL inserts.

● Hash user passwords (e.g. using BCrypt) instead of storing them in plain text, which is currently a known simplification appropriate only for this local learning project.

● Add pagination and more advanced filtering/sorting to the product catalog as it scales.

● Introduce automated unit and integration tests for the DAO and Servlet layers to catch regressions earlier.